

# Warm-up: the $\lambda$ -calculus on paper, then in OCaml

Programming Paradigms  
Session 4 — the warm-up, before the evaluator

Part 1 on paper, in pairs; Part 2 at the keyboard, with `lambda.ml`.

The lecture's block 2 defined the calculus in three forms and one rule. Before we write the evaluator, we run the rule by hand. Nothing here is themed, and in Part 1 there is nothing to run. These are all we need:

---

|                                               |                                     |        |                                                                    |
|-----------------------------------------------|-------------------------------------|--------|--------------------------------------------------------------------|
| $x$                                           | $\lambda x.t$                       | $t\ u$ | a variable; a function of $x$ with body $t$ ; an application       |
| $f\ a\ b = (f\ a)\ b$ ,                       | $\lambda x.x\ y = \lambda x.(x\ y)$ |        | application groups left; a body extends right                      |
| $(\lambda x.t)\ u \rightarrow t[x := u]$      |                                     |        | the rule ( $\beta$ ): the body, with $u$ put in for every free $x$ |
| normal order                                  |                                     |        | the outermost redex first (leftmost among those)                   |
| applicative order                             |                                     |        | the innermost redex first (leftmost among those)                   |
| $\Omega = (\lambda u.u\ u)(\lambda u.u\ u)$ , | $I = \lambda z.z$                   |        | two terms we meet again                                            |

---

Write one  $\beta$ -step per line and underline the redex you reduce.

## Part 1 — on paper

### Exercise 1 ( $\beta$ -reduction)

Reduce to normal form:  $((\lambda x.\lambda y.x\ y)(\lambda z.z\ z))(\lambda w.w)$ .

### Exercise 2 (free, bound, capture)

In  $(\lambda x.\lambda y.x\ y)\ y$ , which occurrences of  $x$  and  $y$  are bound, and which are free? Reduce it. Be careful: the argument is a free  $y$ , and the body contains a  $\lambda y$ .

### Exercise 3 (normal versus applicative order)

Reduce  $(\lambda x.I)\ \Omega$  in normal order. Then start it in applicative order. What happens in each case, and why? (The lecture: normal order finds the normal form whenever one exists; applicative order may not.)

### Exercise 4 (if as a function)

With  $\text{true} = \lambda t.\lambda f.t$ ,  $\text{false} = \lambda t.\lambda f.f$  and  $\text{if} = \lambda c.\lambda t.\lambda e.c\ t\ e$ , reduce  $\text{if}\ \text{true}\ a\ b$  to normal form, then  $\text{if}\ \text{false}\ a\ b$ . Under which order does this work as an `if` should — and what goes wrong under the other, when  $b$  is a term that loops?

### Exercise 5 (a number is a repetition)

With  $1 = \lambda f.\lambda x.f\ x$  and  $\text{succ} = \lambda n.\lambda f.\lambda x.f\ (n\ f\ x)$ , reduce  $\text{succ}\ 1$  to normal form. Which numeral is it, and why does the encoding make that term “two”?

## Part 2 — at the keyboard: `lambda.ml`

The type is the three forms; every function is one `match` on it. `dune exec ./main.exe` reports each hole as *todo*, then *pass* or *FAIL*.

|    | the function                                                 | what the checks want                                            |
|----|--------------------------------------------------------------|-----------------------------------------------------------------|
| W1 | <code>size : term -&gt; int</code>                           | the number of constructors; <code>size Ω = 9</code>             |
| W2 | <code>free_vars : term -&gt; string list</code>              | sorted, no duplicates; $x(\lambda x. x)$ has $x$ free           |
| W3 | <code>is_closed : term -&gt; bool</code>                     | $\Omega$ is closed, $\lambda x. x y$ is not                     |
| W4 | <code>subst : term -&gt; string -&gt; term -&gt; term</code> | Exercise 2 in code: $(\lambda y. x)[x := y]$ is $\lambda y'. y$ |

W4 is the hard one: the `Lam (y, b)` case has two sub-cases, and `fresh` (provided) makes the new name. When W1–W4 pass, Part A of the lab is Exercise 3 in code — `step_cbn` and `step_cbv` on  $(\lambda x. \lambda y. y) \Omega$ .